LAB2 SOLUTIONS

Computer Vision Foundations: Images and Filtering

Ex. 1 - Image Thresholding

```
# Perform binary thresholding using four different values: 30, 60, 90, 120.
# As input image, use a grayscale image.
thresh = [30, 60, 90, 120]
fig, ax = plt.subplots(nrows = 1, ncols = len(thresh), figsize = (3*len(thresh), 3*
    len(thresh)))
for i in range(0, len(thresh)):
    new_lena = np.where(lena_gs <= thresh[i], 0, 255)
    plt.subplot(1,4,(i+1)), plt.imshow(new_lena, cmap=plt.cm.gray), plt.title(f'{
        thresh[i]} thresh'), plt.axis('off')
```

```
# OpenCV2 example
# cv2.threshold(src, thresh, maxval, type[, dst]) --> retval, dst
# It returns the used threshold and the thresholded image.
thresh = [30, 60, 90, 120]
fig, ax = plt.subplots(nrows = 1, ncols = len(thresh), figsize = (3*len(thresh), 3*
    len(thresh)))
for i in range(0, len(thresh)):
    _, new_lena = cv2.threshold(lena_gs, thresh[i], 255, cv2.THRESH_BINARY)
    plt.subplot(1,4,(i+1)), plt.imshow(new_lena, cmap=plt.cm.gray), plt.title(f'{
        thresh[i]} thresh'), plt.axis('off')
```

Ex. 2 - Box Blur (box linear filter)

```
def box_filter(kernel_size):
    '''
    Create a k x k box filter.

    Input parameters
    ----------------
    kernel_size: int
    Size of the squared kernel.

    Output parameters
    ----------------
    Filter: np.array [k x k]
    Box filter

    To check
    ------------
    kernel size must be an odd number
    '''

    assert kernel_size % 2 != 0, "Kernel size must be an odd number."
    kernel = np.ones([kernel_size, kernel_size]) / kernel_size**2
    return kernel
```

```
def conv_2d(image, kernel):
    '''
    Apply a k x k filter to the input image.

    Input parameters
    ----------------
```

```
    image: np.array
    Input image.
    kernel: np.array [k x k]
    Filter to apply

    Output parameters
    ----------------
    Image: np.array

    To check
    ------------
    Grayscale image is required
    '''

    # check grayscale image
    assert len(image.shape) == 2, "Greyscale image is required"

    # add padding to the original image
    kernel_size = kernel.shape[0]
    padding_size = (kernel_size - 1)//2
    image = np.asarray(image,dtype=np.float32)
    img_padded = np.pad(image, (padding_size, padding_size))

    # prepare the empty output image
    output_image = np.zeros_like(image)

    # prepare the filpped kernel for convolution
    flipped_kernel = np.flipud(np.fliplr(kernel))

    # apply convolution to each element of original image
    rows, columns = image.shape
    for row in range(rows):
      for col in range(columns):
           output_image[row,col] = (img_padded[row:row+kernel_size, col:col+
               kernel_size]*flipped_kernel).sum()

    return output_image
```

Ex. 3 - 2-D Gaussian Filter

```
def gaussian_2d(size=11, sigma=1):
    '''
    2-D Gaussian Filter

    Input Parameters
    ----------------
    size: int
      Size of the squared kernel
    sigma: float
      Standard deviation

    Output Parameter
    ----------------
    filter: np.array
      2-D Gaussian Filter
    '''

    # check input parameters
    assert size % 2 != 0, "Kernel size must be an odd number."

    # create the grid (xx, yy)
```

```
    ax = np.linspace(-(size - 1) / 2., (size - 1) / 2., size)
    xx, yy = np.meshgrid(ax, ax)

    # fill the grid with the gaussian values
    kernel = 1/(np.sqrt(2 * np.pi * np.square(sigma))) * np.exp(-0.5 * (np.square(xx)
        + np.square(yy)) / np.square(sigma))

    return kernel / np.sum(kernel)
```

Ex. 4 - Median filter

```
def median_filter(image, kernel_size, padding=True):
    '''
    This function applies the median filter to the input image.

    Input Parameters
    ----------------
    image: np.array
    Input grayscale image
    kernel_size: int
    Dimension of a squared kernel.
    Padding: bool
    If True, the input image is padded with zeros in order to have as output
    an image with the same spatial size of the input image.
    If False, no padded is applied and the output image will be smaller than
    the input one.

    Output Parameters
    -----------------
    image: np.array
    Filtered image

    To check
    ------------
    Grayscale image is required
    kernel size must be an odd number
    '''

    # pass

    # check inputs
    assert len(image.shape) == 2, f"Input image has {image.shape[-1]} channels.
        Grayscale image is required."
    assert kernel_size % 2 != 0, "Kernel size must be an odd number."

    image = np.asarray(image,dtype=np.float32)
    rows, columns = image.shape

    if padding:
        output_image = np.zeros_like(image) # prepare empty output image
        # add padding to the input image
        padding_size = (kernel_size - 1)//2
        img_padded = np.pad(image, (padding_size, padding_size))
        # apply median filter
        for row in range(rows):
            for col in range(columns):
                output_image[row,col] = np.median(img_padded[row:row+kernel_size, col
                    :col+kernel_size])
    else:
        # calculate the dimensions of the image in output
        output_rows = rows - kernel_size + 1
```

```
        output_columns = columns - kernel_size + 1
        output_image = np.zeros([output_rows, output_columns]) # prepare empty output
            image
        # apply median filter
        for row in range(output_rows):
            for col in range(output_columns):
                output_image[row, col] = np.median(image[row:row+kernel_size, col:col
                    +kernel_size])
    return output_image
```

Ex. 5 - Sharpening filter

```
# create blurred image
lena_blur = cv2.GaussianBlur(lena_gs, (21,21), 10)
# cv2.addWeighted is a function that performs addition between two images
# we use it to extract details subtracting the blurred image from the original one (
    step 1)
lena_details = cv2.addWeighted(lena_gs,1,lena_blur,-1,0)
plt.figure(figsize=(10,7))
plt.subplot(2, 3, 1), plt.imshow(lena_gs, cmap=plt.cm.gray), plt.title('Original'),
    plt.axis('off')
plt.subplot(2, 3, 2), plt.imshow(lena_blur, cmap=plt.cm.gray), plt.title('Blur'), plt
    .axis('off')
plt.subplot(2, 3, 3), plt.imshow(lena_details, cmap=plt.cm.gray), plt.title('Details'
    ), plt.axis('off')
# now we use cv2.addWeighted to add the details to the original image (step 2)
for i,k in enumerate([1,2,3]):
    lena_sharp = cv2.addWeighted(lena_gs,1,lena_details,k,0)
    plt.subplot(2, 3, 3+k), plt.imshow(lena_sharp, cmap=plt.cm.gray), plt.title(f'k={
        k}'), plt.axis('off')
```